

Network Dismantling via Reverse Dismantling: Static and Dynamic Algorithms

Jinyu Duan

Guangzhou University
CIAT & Guangdong Key Laboratory
of Industrial Control System Security
& Huangpu Research School
Guangzhou, Guangdong, China
jduan@gzhu.edu.cn

Sijin Wang

Guangzhou University
CIAT & Guangdong Key Laboratory
of Industrial Control System Security
& Huangpu Research School
Guangzhou, Guangdong, China
wsjin@gzhu.edu.cn

Fan Zhang*

Guangzhou University
CIAT & Guangdong Key Laboratory
of Industrial Control System Security
& Huangpu Research School
Guangzhou, Guangdong, China
zhangf@gzhu.edu.cn

Xiang Zhao*

National University of Defense
Technology
National Key Laboratory of Big Data
and Decision
Changsha, Hunan, China
xiangzhao@nudt.edu.cn

Wenjie Zhang

University of New South Wales
School of Computer Science and
Engineering
Sydney, NSW, Australia
wenjie.zhang@unsw.edu.au

Zhihong Tian

Guangzhou University
CIAT & Guangdong Key Laboratory
of Industrial Control System Security
& Huangpu Research School
Guangzhou, Guangdong, China
tianzhong@gzhu.edu.cn

Abstract

For complex networks such as the Web, the Network Dismantling (ND) problem, which asks for the minimum-cost removal of nodes that destroys the giant connected component in the network, is significant in system robustness and misinformation containment. In this paper, we propose a heuristic algorithm, IG+, which is based on reverse dismantling and incorporates novel optimizations. Besides, we design two dynamic algorithms, CCRT-ins and CCRT-rem, employing tree-like indexes to update dismantling results efficiently. Experiments show that our methods outperform state-of-the-art approaches in both effectiveness and efficiency, and can dismantle 10-million-scale networks at arbitrary granularity in a few minutes.

CCS Concepts

• Theory of computation → Dynamic graph algorithms.

Keywords

Graph algorithms; Network dismantling; Dynamic algorithms.

ACM Reference Format:

Jinyu Duan, Sijin Wang, Fan Zhang, Xiang Zhao, Wenjie Zhang, and Zhihong Tian. 2026. Network Dismantling via Reverse Dismantling: Static and Dynamic Algorithms. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792316>

Resource Availability:

The source code of this paper has been made publicly available at <https://doi.org/10.5281/zenodo.17218033>.

*Corresponding authors: Fan Zhang, Xiang Zhao.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792316>

1 Introduction

In real-world systems such as the WWW [28], relatively stable interactive relationships among entities are ubiquitous, giving rise to complex network (graph) structures, e.g., social networks [43], collaboration networks [5], and biological networks [10]. However, numerous studies show that disrupting only a small fraction of critical entities can significantly impair the overall functionality of network systems [12, 16, 25, 40, 41]. Motivated by this observation, several studies [2, 3] focus on how small the cost can be to inflict a specified level of damage on a network. This focus inspires the study of the classical Network Dismantling (ND) problem [9]: given a network (graph) G , selecting a minimum set of nodes (i.e., a dismantling set) to remove such that the nodes of the maximum connected component (MCC) in the residual network are less than c . In these studies, the size of MCC measures the robustness of the system, as connectivity is crucial to the functionality.

ND is widely applied in different applications, especially in the domains of the WWW and social networks. For example, an early study [2] investigates the extent of network fragmentation (i.e., reduction in the size of the MCC) in the WWW under both random errors and targeted attacks. In the domain of social networks, the nodes (i.e., users) selected by the ND problem are often regarded as key information disseminators [17, 38], which means that they have higher influence and should therefore be treated with greater attention [14]. For instance, they are critical for restricting the cascade of misinformation over the network.

Existing Studies. Due to the importance of the ND problem, many studies are dedicated to solving it. Since it is proven that ND is NP-complete [9], most studies adopt heuristic methods. As a typical instance of the Minimum Approximate Submodular Cover problem [11], ND can in principle be solved by the Nemhauser Greedy (NG) algorithm [42], which iteratively removes the node that maximally reduces $|MCC|$ until it drops below c . However, NG is computationally prohibitive on large networks, since each step requires recomputing connectivity after node removal.

To improve efficiency, some studies based on the NG framework propose indirect indicators that are easy to compute to replace exact connectivity evaluation. For example, CI [38], based on optimal percolation theory, defines a new importance score, while FINDER [17] uses the learning method to learn a combined-feature indicator from synthetic data. However, these measures do not strictly follow the objective of minimizing MCC when selecting nodes, which may slightly sacrifice their effectiveness on ND. Despite this, NG-like algorithms still struggle to efficiently handle large graphs, as they require updating the importance of the unselected nodes after each selection. Some methods reduce update frequency to improve efficiency, but the quality of the results will be further sacrificed. More details are given in Sec. 2.

Challenges & Our Solutions. To overcome the above limitations, we observe that solving the ND problem from a reverse dismantling perspective is more promising than NG-like frameworks. Specifically, we adopt an insertion-based framework that sequentially inserts nodes into the graph (and removes them from the dismantling set) following a greedy strategy tied to connectivity, unless the $|MCC|$ constraint is violated. As discussed in Sec. 4, the intuition of reverse dismantling is as follows: maintaining connectivity during node insertions is both far-sighted and sufficiently efficient, eliminating the need for indirect indicators like in the NG-framework.

Although the insertion-based framework is effective, outperforming the well-designed existing solutions is still challenging. On the theoretical side, we show the nonsupermodularity of ND and prove that ND is NP-hard to be approximated within $2 - \epsilon$ for any $\epsilon > 0$ (Thm. 8), which limits the approximation guarantee for heuristic solutions. On the practical side, we propose IG, an Insertion-based Greedy algorithm, which greedily inserts nodes into a sketch graph, and the remaining un-inserted nodes form the result (IG in Sec. 4.1). Meanwhile, we design the Component Graph to integrate and maintain correlations among various connected components, so that $|GCC|$ of the sketch graph can be computed incrementally. To further improve the efficiency, we propose the Delayed Update optimization, which delays the update of the bucket list until the top element changes (IG+ in Sec. 4.2).

In addition, to the best of our knowledge, there is no dynamic algorithm to update the result of ND. We design a light index with a tree structure, denoted as the Connected Component Reconstruct Tree (CCRT), which can be initialized by IG+. Accordingly, two dynamic algorithms, CCRT-ins and CCRT-rem, are proposed based on CCRT to efficiently maintain the result of ND against edge insertions and removals, respectively. The result quality is close to re-executing the best static algorithm (Sec. 5).

Contributions. The contributions are summarized as follows:

- We prove that the ND problem is NP-hard to approximate within $2 - \epsilon$ for any $\epsilon > 0$, and introduce a new insertion-based framework (IG) following the idea of reverse dismantling.
- We design a static algorithm (IG+) that improves efficiency via a bucket-list scoring function with delayed updates.
- We introduce the CCRT index to capture component connectivity, and develop CCRT-ins and CCRT-rem to efficiently handle edge insertions and removals.
- Experiments on 12 datasets show that IG+ outperforms state-of-the-art heuristics by 5x to 2 orders of magnitude. Compared to

re-running static solutions, CCRT-ins and CCRT-rem achieve 1 to 3 orders of speedup with average errors within 0.002.

2 Related Works

Variants of the ND Problem. The dual problem of ND is called the MinMaxC problem [45], which targets at finding a node set V' from G such that $|V'| = b$ and the size of MCC in $G \setminus V'$ is minimized. ND and MinMaxC can be reduced to each other. In addition, there exists a class of problems that examine how network connectivity evolves after node removals, which are collectively referred to as the Critical Node Detection Problem (CNDP), as surveyed in [30]. CNDP seeks a node subset whose removal will degrade certain metrics to the largest extent, e.g., the size of the maximum connected component (MinMaxC) [46, 47], the number of connected components (MaxNum) [7, 8, 36], and the number of connected node pairs (CN) [1, 4, 13, 15, 48–50]. Other variant problems include Weighted Network Dismantling [19], Continuous Network Dismantling [34], etc. Related studies primarily focus on the computational complexity of the problem, the design of heuristic algorithms, and the development of polynomial-time algorithms for specific types of graphs.

Algorithms for the ND Problem. A variety of algorithms are proposed for addressing the ND problem, as surveyed in [3]. Most algorithms can be categorized into the NG-like framework, where nodes are ranked according to some importance functions, and the most critical node is iteratively removed from the graph and added to the result set. Commonly used functions include degree centrality [2], betweenness centrality [22], closeness centrality [32], network entanglement [24], etc. CI [38], based on optimal percolation theory, generally achieves better performance. More recently, learning methods have been employed to learn composite ranking metrics such as FINDER [17], NIRM [52], CLMA [35], and DCRS [53].

A common limitation of all these approaches is that the scores of all remaining nodes should be essentially recomputed and re-ranked after each node removal, since the residual graph changes dynamically. This step is crucial for ensuring the quality of the results, but relatively costly. For instance, for a recent approach, network entanglement [24] (also known as vertex entanglement or VE), the complexity of calculating entanglement can be $O(k^2|V|)$ per vertex, where k is the degree of the vertex. Although some methods mitigate this by reducing the frequency of score updates to balance efficiency and effectiveness, many of them are still hard to scale to large networks.

3 Theoretical Analysis

Let $G = (V, E)$ denote a simple, undirected, and unweighted graph, with $n = |V|$ nodes and $m = |E|$ edges. Given a node $v \in V(G)$, the neighbor set of v in G is denoted by $N(v, G) = \{u \mid u \in V(G) \wedge (u, v) \in E(G)\}$, and the degree of v is denoted as $d(v, G) = |N(v, G)|$. Given a node set $V' \subseteq V(G)$, $G[V']$ denotes the subgraph of G induced by V' . Tab. 1 summarizes the major notations used in the paper. In the following, we define some basic concepts.

Definition 1 (Connected Component). Given an undirected graph G , a subgraph S is a connected component, if (i) S is connected; and (ii) S is maximal, i.e., any supergraph of S is not connected except S itself.

Table 1: Summary of Notations

Notation	Definition
G	an unweighted and undirected graph
$V(G); E(G)$	the vertex/edge set of G
$G[V]$	the subgraph of G induced by vertex set V
n, m	$ V(G) $ and $ E(G) $, respectively
u, v, w	a vertex in the graph
$e, (u, v)$	an edge in the graph
$N(v, G)$	the neighbor set of vertex v in G
$d(v, G)$	the degree of vertex v in G
V'	the vertex cut defined in Def. 2
$MCC(G, V')$	the connected component with the maximum number of vertices in $G[V(G) \setminus V']$
c	the threshold of $ MCC(G, V') $ in ND problem
G^*	the Component Graph to handle different components, defined in Def. 4
V_1, V_2, V_3	a vertex in $V(G^*)$ that represents a non-empty subset of $V(G)$
$score(\cdot)$	the score function to guide node selection, defined in Def. 5
$CCRT_G$	the connected component restructure tree for G

Definition 2 (Vertex Cut). Given a connected graph G , $V' \subseteq V(G)$ is a vertex cut of G if $G[V(G) \setminus V']$ is disconnected.

For a giant component of a network, a vertex cut can divide it into multiple connected components, and we generally focus on the maximum connected component.

Definition 3 (Maximum Connected Component). Given a graph G and a vertex cut V' , the maximum connected component (short for MCC), denoted as $MCC(G, V')$, is the connected component in $G[V(G) \setminus V']$ with the maximum number of nodes.

The Network Dismantling (ND) problem [9] is defined as follows.

Problem Definition. Given a graph G and an integer threshold c , let $f(V') : 2^{[n]} \rightarrow \mathbb{N}$ be a non-negative set function s.t. $f(V') = |MCC(G, V')|$, the network Dismantling (ND) problem is

$$\min\{|V'| : V' \subseteq V(G) \wedge f(V') \leq c\}.$$

In Appx. A.1, we analyze the NP-hardness and inapproximability of the ND problem. We also discuss the theoretical approximation bounds and time complexity bounds of existing heuristic algorithms based on the NG framework for ND.

4 Static Algorithms

As mentioned earlier, the high cost of MCC computation when removing nodes prevents the direct application of the NG algorithm. Therefore, existing studies typically propose indirect indicators to replace the computation of MCC. Although using indirect indicators is a compromise for efficiency at the cost of losing approximation guarantees, their efficiency remains insufficient in large-scale networks. In contrast, maintaining connectivity information while inserting nodes is typically more efficient; for instance, classic data structures like Disjoint Union are available. Based on this insight, we study a new framework for the ND problem: Insert the nodes (and remove them from V') sequentially according to a greedy strategy directly related to connectivity, until $|MCC(G, V')| > c$. The insertion-based framework offers the following advantages:

- Insertion-based algorithms can directly maintain and utilize the impact of insertions on MCC, whereas the removal-based algorithms typically rely on indirect metrics.
- Insertion-based algorithms can incrementally maintain the change in node importance during node selections, whereas the removal-based algorithms cannot calculate the current importance promptly because frequent recalculations are costly.
- Similar to removal-based algorithms, insertion-based algorithms can also generate V' in one batch for arbitrary values of c on a given graph. This batch processing is crucial because, in practical applications, it allows users to select feasible V' and corresponding thresholds c based on their budgets.

We propose IG, an Insertion-based Greedy algorithm, which inserts the nodes in a well-designed greedy manner, and an optimization technique to accelerate IG, forming the IG+ algorithm.

4.1 An Insertion-based Greedy Algorithm (IG)

In this subsection, we propose the IG algorithm. The idea is intuitive: IG progressively removes the nodes from the candidate cut V' and inserts them into the sketch graph $G[V(G) \setminus V']$, unless the insertion will lead to $|MCC(G, V')| > c$. In the above process, IG greedily moves the node that will minimize the increase of $|MCC(G, V')|$.

The IG algorithm is described in Alg. 1. During the initialization phase (Lines 1 - 4), for a good trade-off between effectiveness and efficiency, IG keeps as few nodes as possible in V' while keeping $MCC(G, V') = 1$. Specifically, on line 1, IG initializes V' by $V(G)$ minus a large independent set $MIS(G)$, which ensures $|MCC(G, V')| = 1$. MIS provides the finest start, and it is flexible to merge with any connected components. In our early study, the final $|GCC|$ of our algorithm is not sensitive to the quality of MIS. Therefore, a simple and efficient MIS algorithm [44] is adopted: iteratively include the node $v \in V(G)$ with the smallest $d(v, G)$ and mark the neighbors of v as not selectable.

In the initialization, the independent set $MIS(G)$ forms the sketch graph, where each node is a connected component. Any subsequent insertion will result in the merge of some connected components. To handle the change of merge, we introduce the component graph.

Definition 4 (Component Graph). Given a graph G and a vertex cut V' , the component graph is denoted by $G^* = (V^*, E^*)$ in which V^* is a partition over $V(G)$. A node $V_1 \in V^*$ represents a non-empty subset of $V(G)$ which contains a node in V' or induces a connected component in the sketch graph $G[V(G) \setminus V']$. For E^* , an edge $(V_1, V_2) \in E^*$ iff an edge $(u, v) \in E(G)$ with $u \in V_1$ and $v \in V_2$.

The component graph G^* is initialized in lines 4 - 5, where $V(G^*)$ contains $\{v\}$ for each $v \in V(G)$ and $E(G^*)$ contains $(\{u\}, \{v\})$ for each $(u, v) \in E(G)$. Currently, each connected component in the sketch graph $G[V(G) \setminus V']$ is a single node of $MIS(G)$. Then, we maintain G^* as follows. Whenever we move a node v from V' to the sketch graph (Lines 6-9), a new connected component is formed: we should merge $V_1 \in V(G^*)$ where $V_1 = \{v\}$ with every neighbor $V_2 \in N(V_1, G^*)$ containing the nodes in the sketch graph $G[V(G) \setminus V']$, i.e., satisfying $V_2 \not\subseteq V'$ (Line 12).

We use a Disjoint Set to implement this process and maintain the value $|V_1|$ for the merged V_1 . To maintain the changes in $E(G^*)$, for each $V_3 \in G^*$ that has a common neighbor V_2 with V_1 in G^* , we link

Algorithm 1: IG (with Bucket List)

Input : a graph $G = (V, E)$, an integer c
Output : the vertex cut V' with $|MCC(G, V')| \leq c$
/ Initialize V' , Component graph G^* and $score(\cdot)$ */*
1 $V' \leftarrow V(G) \setminus MIS(G);$
2 $V(G^*) \leftarrow \{V_1 \mid V_1 \subseteq V(G) \wedge |V_1| = 1\};$
3 $E(G^*) \leftarrow \{(V_1, V_2) \mid V_1 \times V_2 \subseteq E(G)\};$
4 $\forall v \in V'$, initialize $score(\{v\})$ by Def. 5;
/ Move V_1 from V' to G^* in each iteration */*
5 **while** $|V'| > 0$ **do**
6 $V_1 \leftarrow \{v\}$ where $v = \arg \min_{v \in V'} score(\{v\})$;
7 **if** $score(\{v\}) > c$ **then**
8 $\quad \text{break};$
9 $V' \leftarrow V' \setminus V_1;$
/ Update G^* and $score(\cdot)$ for moving V_1 */*
10 $E^+ \leftarrow \emptyset;$
11 **for each** $V_2 \in N(V_1, G^*)$ **do**
12 **if** $V_2 \not\subseteq V'$ **then**
13 $\quad \text{Merge } V_1 \text{ and } V_2 \text{ into } V_1 \text{ in } V(G^*) \text{ where } V_1 \leftarrow V_1 \cup V_2;$
14 $\quad \text{for each } V_3 \in N(V_2, G^*)$ **do**
15 $\quad \quad E^+ \leftarrow E^+ \cup \{(V_1, V_3)\};$
16 $\quad \quad score(V_3) \leftarrow score(V_3) - |V_2|;$
17 $\quad \text{Remove } V_2 \text{ and its incident edges from } G^*;$
18 $E(G^*) \leftarrow E(G^*) \cup E^+;$
19 **for each** $V_2 \in N(V_1, G^*)$ **do**
20 $\quad score(V_2) \leftarrow score(V_2) + |V_1|;$
21 $\quad \text{Modify } V_2 \text{ in the bucket list with new } score(V_2);$
22 **return** $V';$

V_1 and V_3 in G^* (Lines 10, 14-15 and 18). Note that V_2 is a connected component in the sketch graph, so we have $V_3 \subseteq V' \wedge |V_3| = 1$.

To minimize the incremental size of $|GCC|$, the algorithm selects $V_1 = v$, where $v \in V'$, for insertion to minimize the size of the newly generated connected component C_v . Clearly, C_v is induced by $V_1 = \{v\}$ and V_1 's every neighbor V_2 containing the nodes in the sketch graph. We formally define this as the insertion score of V_1 .

Definition 5 (Insertion Score). For each node $v \in V'$, let $V_1 = \{v\}$, we have $score(V_1) = 1 + \sum_{V_2 \in N(V_1, G^*) \wedge V_2 \not\subseteq V'} |V_2|$.

The maintenance of $score(\cdot)$ is decided by the changes of connected components in the sketch graph:

a) For each $V_3 \in G^*$ that has a common neighbor V_2 with V_1 in G^* , we first decrease $score(V_3)$ by $|V_2|$ at line 16 since V_2 will be removed at line 17;

b) Due to the generation of a new connected component V_1 , $\forall V_2 \in N(V_1, G^*)$, $score(V_2)$ increases by $|V_1|$ (Lines 19-21).

Implementation and Complexity Analysis. Based on the following property, Alg. 1 can be implemented with bucket list [21].

PROPERTY 1. $score(\cdot)$ is an integer in the range of 1 to n . Inserting any set of nodes into the sketch graph $G[V(G) \setminus V']$ will not decrease the $score(\cdot)$ of others.

LEMMA 1. The complexity of a pop, top, increase, or push operation on the bucket list used in the IG framework is $O(1)$.

Due to Lemma 1, IG has the following time complexity.

THEOREM 1. The time complexity of the IG algorithm is $O(c \cdot m)$.

PROOF. The proof is provided in Appx. A.2. $\square$

As the problem generally sets a relatively small MCC threshold, i.e., c is typically small, the IG algorithm always meets practical requirements. Even without this assumption, the domination of the time cost lies in enumerating the two-hop neighbors of nodes, which is generally efficient in typical networks, e.g. in scale-free networks that has a degree distribution $P(d) \propto d^{-\alpha}$, in which $2 < \alpha < 3$, the complexity is $O(n^{2/(\alpha-1)})$.

4.2 Delayed Update and IG+ Algorithm**Algorithm 2:** IG+

Input : a graph $G = (V, E)$, an integer c
Output : the vertex cut V' with $|MCC(G, V')| \leq c$
1 Execute Lines 1 - 4 in Alg. 1;
2 **while** $|V'| > 0$ **do**
3 $V_1 \leftarrow \{v\}$ where v is the top element in the bucket list;
/ Delayed Update */*
4 **while** $score_{min} < score(V_1)$ **do**
5 $\quad \text{Modify } v \text{ in the Bucket List with } score(\{v\});$
6 $\quad V_1 \leftarrow \{v\}$ where v is at the top of the bucket list;
7 Execute Lines 7 - 18 in Alg. 1;
8 **for each** $V_2 \in N(V_1, G^*)$ **do**
9 $\quad score(V_2) \leftarrow score(V_2) + |V_1|;$
10 **return** $V';$

Although the bucket list operations are efficient in Alg. 1, excessive modifications can still introduce significant computational overhead. Furthermore, many updates are redundant and do not influence the decisions of the algorithm. Specifically, the following factors are observed: a) the correctness of Alg. 1 depends on the fact that the selected v on the top of the bucket list has the minimum $score(v)$; b) according to Prop. 1, updating the score of any node v does not affect the top element unless v is the current top element.

Therefore, we propose the Delayed Update optimization. When the $score(\cdot)$ of a node changes, we record its new score without immediately updating it in the bucket list, unless the node becomes the top element. As stated in Prop. 1, inserting any nodes into $G[V \setminus V']$ can only increase or preserve the $score(\cdot)$ values of other nodes, the above optimization reduces the number of modification operations while ensuring that each element added to $G[V \setminus V']$ retains the minimum $score(\cdot)$.

IG+ Algorithm. Alg. 2 shows the pseudocode of IG+, i.e., the IG algorithm with the Delayed Update optimization. The difference between IG+ and IG is as follows. Firstly, the nodes in the bucket list are not modified immediately after the score update in lines 8-9. Secondly, at lines 4 - 6, when the top element of the bucket list is queried, and its score recorded in the bucket list (i.e., $score_{min}$) is incorrect, we update the bucket list and re-query the top element.

5 Maintenance Algorithms

In this section, we propose CCRT (Def. 6), a tree-based index, designed to efficiently retrieve $|MCC|$ after removing (cutting) a subset V' from a given graph G . It is used to quickly compute the ND problem for any given threshold c . Moreover, due to the incremental maintainability of the index, we propose the CCRT-ins and CCRT-rem algorithms to efficiently update the result of ND against edge insertions and removals, respectively, based on CCRT.

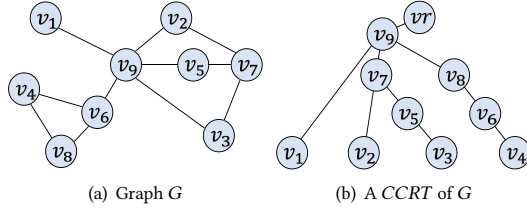


Figure 1: Example of the CCRT Index

Definition 6 (Connected Component Restructure Tree). Given a graph G , the Connected Component Restructure Tree of G , denoted as $CCRT_G$ (or $CCRT$ when context is clear), is defined as a rooted tree with node set $V(G) \cup \{vr\}$, satisfying the following conditions: i) vr is the root of the tree; ii) for any $v \in V(G)$, let $CCRT_G.subtree(v)$ denote the subtree of $CCRT_G$ rooted at v , the induced subgraph of G over the node set $V(CCRT_G.subtree(v))$ is connected; iii) for any $v \in V(G)$ and any neighbor $u \in N(v, G)$, the node u must either be in the subtree of v or be an ancestor of v in the $CCRT_G$.

Algorithm 3: $CCRT_G.solve(\cdot)$

Input : $CCRT_G$, an integer c
Output : V' with $|MCC(G, V')| \leq c$
1 $V' \leftarrow \emptyset, Cset \leftarrow CCRT_G.cld_{vr}$;
2 **while** $|MCC(G, V')| > c$ **do**
3 $v \leftarrow \operatorname{argmax}_{v \in Cset} |CCRT_G.subtree_v|$;
4 $V' \leftarrow V' \cup \{v\}$;
5 $Cset \leftarrow (Cset \cup CCRT_G.cld_v) \setminus \{v\}$;
6 **return** V' ;

Fig. 7(b) illustrates a CCRT of the graph G shown in Fig. 7(a).

Suppose we have a $CCRT_G$, for any threshold c , the method $CCRT_G.solve(c)$ (Alg. 3) can be invoked to output a solution V' for ND from the given $CCRT_G$. Specifically, the algorithm begins by inserting all child nodes of the root vr into the candidate set $Cset$ (Line 1). Then, in each step, it selects v in $Cset$ whose corresponding subtree $CCRT_G.subtree_v$ has the largest size (Line 3). The selected node v is then moved from $Cset$ to V' , and all of its child nodes are added to $Cset$ (Lines 4-5). This process repeats unless the corresponding subtrees for all the nodes in $Cset$ have sizes no larger than c (Line 2) and V' is a solution from the given $CCRT_G$.

THEOREM 2. $CCRT_G.solve(c)$ runs in $O(\sum_{v \in V'} N(v, G) + n - c) \leq O(m)$ time, where V' is its result.

PROOF. Since the nodes entering $Cset$ are neighbors of the nodes in V' , and using the bin sort allows the total complexity of argmax to be $O(n - c)$, the complexity is as stated in the theorem. $\square$

Note that a graph G may correspond to different $CCRT_G$ structures. Thus, Alg. 3 may produce different results where the result quality (the size of MCC) depends on the input $CCRT$.

For a given G , Alg. 4 constructs an effective $CCRT_G$ using IG+. Specifically, once IG+ removes a node v from V' and merges its corresponding set V_1 in G^* with its neighbors in $V \setminus V'$, the construction process simultaneously builds the subtree of $CCRT$ corresponding to the new V_1 (i.e., V_1 after merging). Since each $V_2 \in G^*$ contained in $V \setminus V'$ has already been processed before the merging of v for

Algorithm 4: Construct $CCRT$ with IG+

Input : a graph $G = (V, E)$
Output : a $CCRT_G$ of G
1 Execute Lines 1 - 4 in Alg. 1;
2 **for each** $v \in V(G) \cup \{vr\}$ **do**
3 $CCRT_G.pa_v = null, CCRT_G.cld_v = \emptyset$;
4 **while** $|V'|$ **do**
5 Execute Lines 3 - 6 in Alg. 2;
6 $E^+ \leftarrow \emptyset$;
7 **for each** $V_2 \in N(V_1, G^*)$ **do**
8 **if** $V_2 \notin V'$ **then**
9 $u \leftarrow CCRT_G.root_{V_2}$;
10 $CCRT_G.pa_u \leftarrow v$;
11 $CCRT_G.cld_u \leftarrow CCRT_G.cld_u \cup \{u\}$;
12 Execute Lines 13 - 17 in Alg. 1;
13 $E(G^*) \leftarrow E(G^*) \cup E^+$;
14 Execute Lines 8 - 9 in Alg. 2;
15 $/* vr$ is the virtual root of $CCRT_G$ $*/$
16 $CCRT_G.root_{V \cup \{vr\}} \leftarrow vr$;
17 **for each** $v \in V$ **do**
18 **if** $CCRT_G.pa_v = null$ **then**
19 $CCRT_G.pa_v \leftarrow vr$;
20 $CCRT_G.cld_{vr} \leftarrow CCRT_G.cld_{vr} \cup \{v\}$;
21 **return** $CCRT_G$;

the new V_1 , constructing the tree for V_1 simply requires linking the trees of its neighboring components (i.e., those corresponding to the neighbors in $V \setminus V'$) as subtrees of the new node v (Lines 7–11). As the algorithm eventually removes all nodes from V' (Line 4), each resulting $V_1 \in V(G^*)$ corresponds to a connected component in G , with its tree fully constructed. The final step is to link each of the above trees to the root node vr in $CCRT_G$ (Lines 15–19).

THEOREM 3. The output of Alg. 4 is a $CCRT$.

PROOF. The proof is provided in Appx. A.2. $\square$

Meanwhile, the following property guarantees the quality of the solution produced by $CCRT_G.solve(\cdot)$ when $CCRT_G$ is built by IG+.

PROPERTY 2. If we construce $CCRT_G$ of G by Alg. 4, for any given c , we have $|CCRT_G.solve(c)| = |IG+(G, c)|$.

PROOF. The proof is provided in Appx. A.2. $\square$

The core idea of our proposed solution to the ND problem on dynamic graphs lies in dynamically maintaining the $CCRT$ index under both edge insertion and removal scenarios, and invoking $CCRT_G.solve(\cdot)$ to answer the query. The problem of maintaining the $CCRT$ index is defined as follows.

Definition 7 ($CCRT$ Maintenance Problem). Given a set of edges E^+ (resp. E^-), a graph G , and an index $CCRT_G$, the problem of $CCRT$ maintenance is to update the $CCRT$ index after inserting E^+ into G (resp. removing E^- from G).

Next, we introduce two algorithms, $CCRT$ -ins and $CCRT$ -rem, designed to update the $CCRT$ index with edge insertion and edge removal, respectively. The intuition is to locally identify parts of the current $CCRT$ that do not meet its definition.

Although the updated $CCRT$ still satisfies Def. 6, it may not be the same as the $CCRT$ reconstructed from scratch on the updated graph using Alg. 4. However, as demonstrated in the experiments

Algorithm 5: CCRT-ins

Input : $G, CCRT_G$, an edge (u, v)
Output : $CCRT_{G'}$, where $G' = G(V(G), E(G) \cup \{(u, v)\})$

```

1  $CCRT_{G'} \leftarrow CCRT_G, G' \leftarrow G(V(G), E(G) \cup \{(u, v)\});$ 
2  $w \leftarrow null;$ 
3 while  $u \neq v$  do
4   if  $w \neq null$  then
5      $pa \leftarrow CCRT_{G'}.pa_w;$ 
6      $CCRT_{G'}.cld_{pa} \leftarrow CCRT_{G'}.cld_{pa} \setminus \{w\};$ 
7      $CCRT_{G'}.pa \leftarrow null;$ 
8   if  $|CCRT_{G'}.subtree_u| < |CCRT_{G'}.subtree_v|$  then
9      $CCRT_{G'}.cld_u \leftarrow CCRT_{G'}.cld_u \setminus \{w\};$ 
10     $CCRT_{G'}.pa_w \leftarrow u;$ 
11     $w \leftarrow u, u \leftarrow CCRT_{G'}.pa_u;$ 
12   else
13      $CCRT_{G'}.cld_v \leftarrow CCRT_{G'}.cld_v \setminus \{w\};$ 
14      $CCRT_{G'}.pa_v \leftarrow v;$ 
15      $w \leftarrow v, v \leftarrow CCRT_{G'}.pa_v;$ 
16 return  $CCRT_{G'};$ 
```

in Sec. 6, after thousands of edge modifications, the result quality of ND corresponding to the updated $CCRT$ remains very close to that of the reconstructed $CCRT$.

5.1 Maintenance on Edge Insertions

Given $CCRT_G$ and an edge to be inserted (u, v) , let $G' = G \cup \{(u, v)\}$. Since any connected subgraph of G remains connected in G' , the existing $CCRT_G$ satisfies the condition (ii) of Def. 6. However, as nodes u and v become neighbors in G' , the algorithm should locally modify the structure of $CCRT_G$ to further satisfy condition (iii).

Consequently, we propose CCRT-ins: for the end-point u of the inserted edge (u, v) that is the root of the smaller subtree compared with the other end-point v , it takes the subtree rooted at u as a child of v . Then we reorder and reconnect their ancestors from bottom to top in a specific order.

The detailed procedure is presented in the pseudocode of Alg. 5. Specifically, for each newly inserted edge (u, v) , the algorithm initializes two pointers, one pointing to node u and the other to node v . These pointers then traverse and process all ancestors of u and v until reaching their least common ancestor (Line 3). During each iteration, the algorithm detaches the already processed node from the tree (Lines 4 - 7) and compares the sizes of the subtrees rooted at the current nodes pointed to by the two pointers (Line 8). Suppose $|CCRT_{G'}.subtree_u| < |CCRT_{G'}.subtree_v|$, node u is connected as the parent of the last processed node, and the pointer for u is moved to $CCRT_{G'}.pa_u$ (Lines 9 - 11).

Analysis. We analyze the correctness and complexity of the algorithm as follows.

THEOREM 4. *For $G' = G(V(G), E(G) \cup \{(u, v)\})$, the output of Alg. 5 satisfies Def. 6.*

PROOF. The proof is provided in Appx. A.2. $\square$

THEOREM 5. *When inserting (u, v) to G , Alg. 5 runs in $O(dep(u) + dep(v) - 2dep(lca(u, v)))$ time, where $dep(\cdot)$ is the depth of the node in $CCRT_G$ and $lca(u, v)$ is the least common ancestor of u and v .*

PROOF. The proof is provided in Appx. A.2. $\square$

Algorithm 6: CCRT-Rem

Input : $G, CCRT_G$, an edge set $E^- \subseteq G$
Output : $CCRT_{G'}$, where $G' = G(V(G), E(G) \setminus E^-)$

```

1  $\forall v \in V(G) \cup \{vr\}, Dsu(v) \leftarrow \{v\};$ 
2  $CCRT_{G'} \leftarrow CCRT_G, G' \leftarrow G(V(G), E(G) \setminus E^-);$ 
3  $dfsCCRT(vr, CCRT_{G'}, G', Dsu(\cdot));$ 
4 return  $CCRT_{G'};$ 
5 Procedure  $dfsCCRT(v, CCRT_{G'}, G', Dsu(\cdot))$ 
6    $v$  is in the stack;
7   for each  $u \in CCRT_{G'}.cld_v$  do
8      $dfsCCRT(u, CCRT_{G'}, G', Dsu(\cdot));$ 
9      $CCRT_{G'}.pa_u \leftarrow null$ 
10   $CCRT_{G'}.cld_v \leftarrow \emptyset;$ 
11  if  $v \neq vr$  then
12    for each  $u \in N(v, G')$  s.t.  $u$  is not in the stack do
13       $w \leftarrow CCRT_{G'}.root_{Dsu(u)};$ 
14      if  $w \neq v$  then
15         $CCRT_{G'}.cld_v \leftarrow CCRT_{G'}.cld_v \cup \{w\};$ 
16         $CCRT_{G'}.pa_w \leftarrow v;$ 
17        Merge  $Dsu(v)$  and  $Dsu(w);$ 
18  else
19    for each  $u \in G'$  s.t.  $CCRT_{G'}.pa_u = null$  do
20       $CCRT_{G'}.cld_v \leftarrow CCRT_{G'}.cld_v \cup \{u\};$ 
21       $CCRT_{G'}.pa_u \leftarrow v;$ 
22   $v$  is not in the stack;
23 return;
```

5.2 Maintenance on Edge Removals

Maintaining $CCRT$ in the case of edge removal is usually more challenging compared to edge insertions. Such maintenance is intuitively unbounded, i.e., any maintenance algorithm may inevitably need to traverse portions of the graph that are unrelated to the given modification. Consequently, we propose a batched maintenance algorithm, CCRT-rem, which requires only a single traversal of the $CCRT$ to complete the update for any number of edge removals.

The detailed procedure is presented in the pseudocode of Alg. 6. Specifically, let $G' = G(V(G), E(G) \setminus E^-)$, where E^- is the set of edges to be removed. CCRT-rem first initializes a disjoint-set union structure $Dsu(\cdot) : V(G) \cup \{vr\} \rightarrow V(G) \cup \{vr\}$ and updates the graph (Lines 1–2). Then it invokes $dfsCCRT(\cdot)$, to traverse and modify the current $CCRT$ (Line 3).

As shown in Alg. 6, for each $v \in V(G) \cup vr$, the dfs algorithm first updates the structure of the subtree rooted at v , followed by reconstructing the relationships between v and the nodes within its subtree (Lines 7–9). If $v \neq vr$, the update strategy involves enumerating each $u \in N(v, G')$, identifying the root w of the tree structure that currently contains u , and connecting w as a child of v (Lines 12–17). The operation for finding w can be performed efficiently by maintaining the disjoint-set union structure $Dsu(\cdot)$. Finally, for the root node vr , similar to Alg. 4, it connects all the nodes that currently do not have a parent and takes them as child nodes (Lines 19–21).

Analysis. We analyze the correctness and complexity of the algorithm as follows.

THEOREM 6. *For $G' = G(V(G), E(G) \setminus E^-)$, the output of the CCRT-rem algorithm satisfies Def. 6.*

PROOF. The proof is provided in Appx. A.2. $\square$

THEOREM 7. *The time complexity of Alg. 6 to remove E^- into G is $O((n + m)\alpha(n))$, where $\alpha(n)$ denotes the amortized complexity of disjoint set operations.*

PROOF. The proof is provided in Appx. A.2. $\square$

Table 2: Statistics of Datasets

Dataset	n	m	α	d_{max}	d_{min}	Type
Crime	829	1,473	2.65	25	3.55	CR
HI-II-14	4,165	13,087	2.30	286	6.28	BI
Digg	29,652	84,781	2.24	283	5.72	SO
Gnutella31	62,561	147,878	2.61	95	4.72	CO
Enron	33,696	180,811	2.19	1,383	10.73	CO
Epinions	75,877	405,739	2.12	3,044	10.69	SO
Facebook	63,392	816,831	2.21	1,098	25.77	SO
Youtube	1,134,890	2,987,624	2.35	28,754	5.26	SO
Flickr	1,624,991	15,473,043	2.07	27,224	19.04	SO
BAFlickr	2,000,000	18,848,124	2.06	30,028	18.84	SY
Superuser	192,409	714,570	2.22	14,294	3.71	DY
Wiki	2,863,439	8,146,544	2.87	146,311	5.69	DY

6 Experimental Evaluation

6.1 Experiment Setting

Algorithms. We evaluate the following algorithms.

- (1) **CI [20, 39]:** the SOTA algorithm based on a function that finds V' greedily. As in their shared code, the parameters *ballRadius* and *updateBatch* are set to 2 and 10, respectively.
- (2) **FINDER [17]:** the SOTA algorithm based on deep reinforcement learning that finds V' according to the learned indicator.
- (3) **NIRM [52]:** the adaptive version of NIRM aims to find V' using the GAT network, while the efficiency is not the focus.
- (4) **VE [24]:** the algorithm finds V' via Vertices Entanglement. Although VE is an innovative centrality metric, its computational efficiency is not superior.
- (5) **IG+, IG-B, IG:** our proposed static algorithm and its variants.
- (6) **CCRT-ins, CCRT-rem:** our proposed maintenance algorithms to update the vertex cut set (Alg. 5 for edge insertions and Alg. 6 for edge removals, respectively, both equipped with Alg. 3).

We use the open-source codes of FINDER [18], CI [54], NIRM [51] and VE [23]. The above codes are the exhaustive versions, setting $c = 0$ by default and removing all the nodes in G in an order according to their cut power, which also computes results for all values of c since $\forall c' > c'' \geq 0$, the output for input c' is a prefix of the output for input c'' .

For a fair comparison, we use the exhaustive version of IG+ that computes results for all values of c , i.e., set $c = n$ since our algorithms are based on node insertion.

Datasets. 12 datasets are used in the experiments, including all the real-world datasets in FINDER [17], 2 dynamic datasets (Su[31] and Wi[29]) to evaluate our dynamic algorithms, and a synthetic dataset (BAFlickr) generated from Flickr using the Barabási–Albert model[6], where the parameter m controlling the average degree is set to the average degree of Flickr. The maximum one has approximately 20M edges. Table 2 presents the details of the datasets. We

use CR, BI, SO, CO, SY, and DY to denote criminal networks, biological networks, social networks, collaboration networks, synthetic networks, and dynamic networks, respectively. α is a parameter used to characterize the degree distribution of each dataset. Note that since most real-world networks are only approximately scale-free [37], we restrict our discussion to the portion of the degree distribution where d is larger than a certain threshold x_{min} .

Executions. Our algorithms are implemented using C++ and compiled with GCC 9.4.0 under O3 optimization. The compared open-source codes are executed in the recommended setting. The computer has an Intel Xeon 2.10GHz CPU and 512G of RAM.

6.2 Experiment Result

Exp-1 Result Quality. As discussed above, we follow the setting in FINDER [17] to exhaustively consider all algorithms for computing a sequence of node insertions or removals. Specifically, in each sequence, we plot the line graph between $|V'|$ and all different c from 0 to $|V(G)|$. Note that since the vertex cut satisfying a large c here is always contained in the vertex cut satisfying a small c for the same algorithm, all the algorithms can solve the ND problem for all given c in one exhaustive computation.

We show the line graphs of all datasets in Figs. 2 & 8, which compare the performance of IG+, and competitors for any threshold c . In all data sets, IG+ achieves the desired $|MCC|$ with a smaller budget, effectively dismantling the original graph G to meet the most specified thresholds. In particular, for the smaller thresholds we are concerned with, i.e., $20\%|V(G)|$, $10\%|V(G)|$, and $1\%|V(G)|$ as shown in Appr. C. Note that NIRM and VE do not return on 7 datasets and 8 datasets, and CI does not return on the BAFlickr dataset, respectively, due to exceeding the time limit (12 hours) or running out of memory.

Interestingly, there is a sharp point in each line of IG+ when $|V'|$ is moderate. From a decreasing view on $|V'|$, the insertion of IG+ increases $|MCC|$ slowly at the early stage and then forms one large MCC quickly, which may inevitably connect many large components. Then, the MCC grows relatively slowly since most components are already connected now. Meanwhile, from an increasing view on $|V'|$, this reflects that the networks rapidly fragmentate when the removed nodes exceed a critical value [2]. Besides, a larger c is easier for removal-based algorithms since fewer nodes should be removed, while IG+ performs comparatively well with significantly better efficiency.

Exp-2 Algorithm Efficiency. We evaluate the time cost of the algorithms (except NIRM and VE as they cannot be applied to large datasets). As shown in Fig. 3(a), when the algorithms compute results for all values of c exhaustively, IG+ is at least 5x faster than the baselines on all datasets except Flickr and BAFlickr, in which IG+ is about 3x faster, e.g., on Youtube, IG+ is about 8x faster than FINDER (25s vs. 207s). In addition, as shown in Fig. 3(b), we also test the algorithms with the setting $c = 0.5 \cdot |V|$. Note that, except for IG+, the algorithms do not support early termination based on the given c in practical applications since they do not have the $|GCC|$ computations. In the experiments, we have to use the stop point regarding c inferred from the exhaustive results.

We also evaluate the scalability of IG+ and the effectiveness of optimizations in Appx. C.

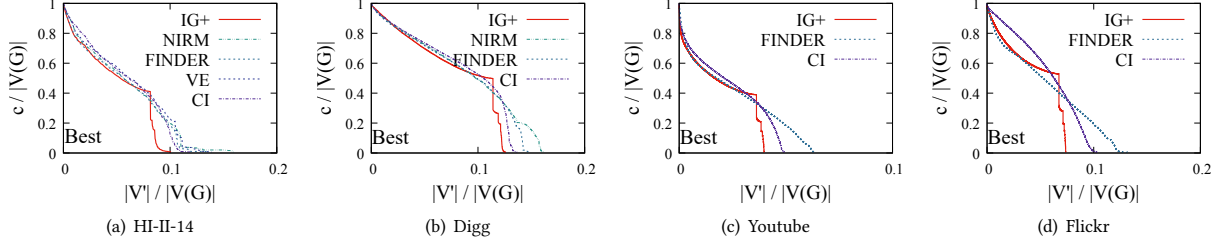
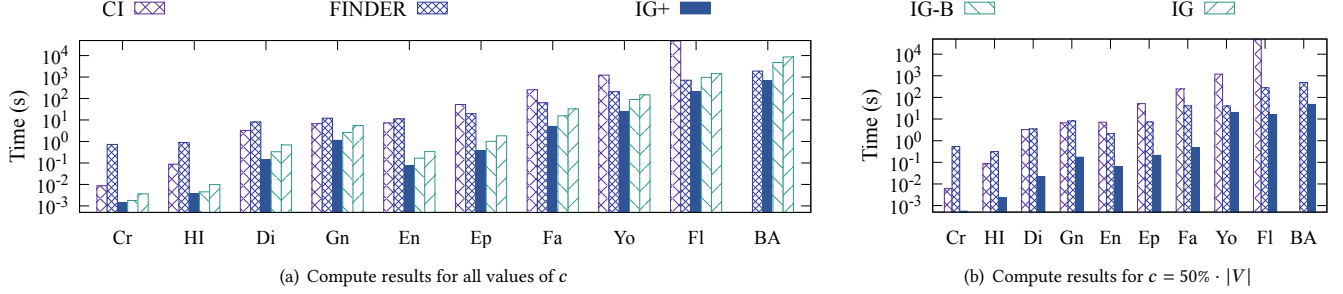
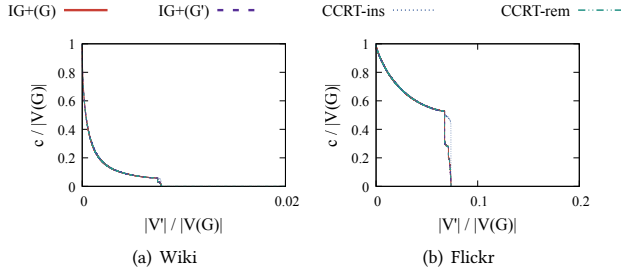
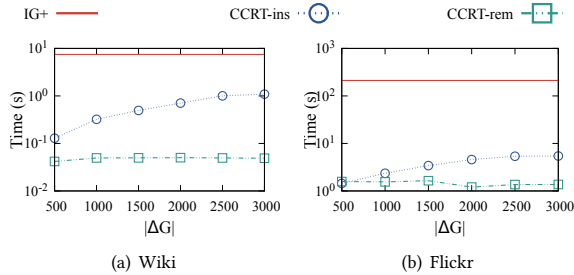
Figure 2: $|V'|$ of Different Algorithms when Threshold c Changes

Figure 3: Time Costs of IG Algorithms and Competitors on All Datasets

Figure 4: Results after Inserting/Removing E^* ($|E^*| = 3000$)Figure 5: Maintenance Time with Different $|\Delta G|$

Exp-3 Performance of CCRT-ins and CCRT-rem. We evaluate our maintenance algorithms using 2 dynamic datasets and 2 large static datasets. On the latter, we simulate dynamic experiments through random edge removals and insertions.

Figs. 4 & 10 compare $|V'|$ under different c across four settings: IG+ on G and G' (where $G' = G \setminus E^*$ and $|E^*| = 3000$, and for dynamic networks, E^* is the set of edges with the latest timestamp), and maintenance algorithms CCRT-ins (inserting E^* into G') and CCRT-rem (removing E^* from G) equipped with $CCRT.solve(\cdot)$.

The results of both CCRT-ins and CCRT-rem closely match the outputs of static algorithms. The maximum average error is 0.0015 (CCRT-ins vs. IG+ (G) on Flickr) and 0.0001 (CCRT-rem vs. IG+ (G') on Superuser), respectively, which mainly occurs at a moderate c due to limited candidate exploration compared with the static IG+.

Figs. 5 & 10 compare the runtimes by varying $|\Delta G|$. IG+ denotes the runtime of exhaustive computation for every c , while CCRT-ins and CCRT-rem include the runtime of updating the index and exhaustive solving the problem with $CCRT.solve(\cdot)$ (Alg. 3), i.e., computing the results of every c . The maintenance methods, CCRT-ins and CCRT-rem, achieve 1–2 orders of magnitude speedup. Even when the number of edge modifications reaches 3,000, the dynamic algorithms still demonstrate a significant speedup compared to the static algorithm. The runtime of CCRT-rem is nearly constant, aligning with its time complexity in Thm. 7.

7 Conclusion

In this paper, we study the ND problem. We prove that ND is NP-hard to approximate, and find that the removal framework in existing solutions is less effective than the insertion framework. Based on this, we propose IG, an insertion-based algorithm, and its optimized variant IG+ with a delayed bucket-list update. Moreover, based on a lightweight index, we propose efficient algorithms for maintaining the results of ND against graph dynamics.

Acknowledgments

This work is partially supported by the National Natural Science Foundation of China (62572140, U25B2047, U2436208, 62536007, 62272469), the Guangdong Basic and Applied Basic Research Foundation (2024A1515011501), the Guangdong S&T Programme under Grant 2024B0101010002, and the Project of Guangdong Key Laboratory of Industrial Control System Security (2024B1212020010). We thank the anonymous reviewers for their insightful suggestions.

References

- [1] Bernardetta Addis, Marco Di Summa, and Andrea Grosso. 2013. Identifying critical nodes in undirected graphs: Complexity results and polynomial algorithms for the case of bounded treewidth. *Discret. Appl. Math.* 161, 16–17 (2013), 2349–2360.
- [2] Réka Albert, Hawoong Jeong, and Albert-László Barabási. 2000. Error and attack tolerance of complex networks. *nature* 406, 6794 (2000), 378–382.
- [3] Oriol Artime, Marco Grassia, Manlio De Domenico, James P Gleeson, Hernán A Makse, Giuseppe Mangioni, Matjaž Perc, and Filippo Radicchi. 2024. Robustness and resilience of complex networks. *Nat. Rev. Phys.* 6, 2 (2024), 114–131.
- [4] Ashwin Arulselvan, Clayton W. Commander, Lily Eleftheriadou, and Panos M. Pardalos. 2009. Detecting critical nodes in sparse graphs. *Comput. Oper. Res.* 36, 7 (2009), 2193–2200.
- [5] A.L. Barabási, H. Jeong, Z. Néda, E. Ravasz, A. Schubert, and T. Vicsek. 2002. Evolution of the social network of scientific collaborations. *PHYSICA A* 311, 3 (2002), 590–614.
- [6] Albert-László Barabási and Réka Albert. 1999. Emergence of Scaling in Random Networks. *Science* 286, 5439 (1999), 509–512.
- [7] André Berger, Alexander Grigoriev, and Ruben van der Zwaan. 2014. Complexity and approximability of the k -way vertex cut. *Networks* 63, 2 (2014), 170–178.
- [8] Hans L. Bodlaender. 1988. Some Classes of Graphs with Bounded Treewidth. *Bull. EATCS* 36 (1988), 116–125.
- [9] Alfredo Braunstein, Luca Dall'Asta, Guilhem Semerjian, and Lenka Zdeborová. 2016. Network dismantling. *PNAS* 113, 44 (2016), 12368–12373.
- [10] Ed Bullmore and Olaf Sporns. 2009. Complex brain networks: graph theoretical analysis of structural and functional systems. *Nat. Rev. Neurosci.* 10, 3 (2009), 186–198.
- [11] Abhimanyu Das and David Kempe. 2018. Approximate Submodularity and its Applications: Subset Selection, Sparse Approximation and Dictionary Selection. *J. Mach. Learn. Res.* 19 (2018), 3:1–3:34.
- [12] Manlio De Domenico and Alex Arenas. 2017. Modeling structure and resilience of the dark network. *Phys. Rev. E* 95, 2 (2017), 022313.
- [13] Amaro de Sousa, Juan Piccini, Franco Robledo, and Pablo Romero. 2019. An Interplay between Critical Node Detection and Epidemic Models. In *RNDM*. 1–7.
- [14] Michela Del Vicario, Alessandro Bessi, Fabiana Zollo, Fabio Petroni, Antonio Scala, Guido Caldarelli, H Eugene Stanley, and Walter Quattrociocchi. 2016. The spreading of misinformation online. *PNAS* 113, 3 (2016), 554–559.
- [15] Thang N. Dinh, Ying Xuan, My T. Thai, E. K. Park, and Taieb Znati. 2010. On Approximation of New Optimization Methods for Assessing Network Vulnerability. In *INFOCOM*. 2678–2686.
- [16] John C Doyle, David L. Alderson, Lun Li, Steven Low, Matthew Roughan, Stanislav Shalunov, Reiko Tanaka, and Walter Willinger. 2005. The “robust yet fragile” nature of the Internet. *PNAS* 102, 41 (2005), 14497–14502.
- [17] Changjun Fan, Li Zeng, Yizhou Sun, and Yang-Yu Liu. 2020. Finding key players in complex networks through deep reinforcement learning. *Nat. Mach. Intell.* 2, 6 (2020), 317–324.
- [18] Changjun Fan, Li Zeng, Yizhou Sun, and Yang-Yu Liu. [n.d.]. FINDER. <https://github.com/FFrankyy/FINDER>.
- [19] Zhidan Feng, Huimin Song, and Xingqin Qi. 2024. A novel algorithm for the generalized network dismantling problem based on dynamic programming. *Chaos, Solitons & Fractals* 180 (2024), 114585.
- [20] Zhu Fengkuangtian. 2018. Improved collective influence of finding most influential nodes based on disjoint-set reinsertion. *Sci. Rep.* 8, 1 (2018), 14503.
- [21] Charles M. Fiduccia and Robert M. Mattheyses. 1982. A linear-time heuristic for improving network partitions. In *DAC*. 175–181.
- [22] Linton C Freeman. 1977. A set of measures of centrality based on betweenness. *Sociometry* (1977), 35–41.
- [23] Yiming Huang, Hao Wang, Xiao-Long Ren, and Linyuan Lü. [n.d.]. VertexEntanglement. <https://github.com/Yiminghh/VertexEntanglement>.
- [24] Yiming Huang, Hao Wang, Xiao-Long Ren, and Linyuan Lü. 2024. Identifying key players in complex networks via network entanglement. *Commun. Phys.* 7, 1 (2024), 19.
- [25] Goodwin Jazmin. 2020. Major internet outage: Dozens of websites and apps were down.
- [26] Subhash Khot. 2002. On the power of unique 2-prover 1-round games. In *STOC*. 767–775.
- [27] Subhash Khot and Oded Regev. 2008. Vertex cover might be hard to approximate to within 2-epsilon. *J. Comput. Syst. Sci.* 74, 3 (2008), 335–349.
- [28] Jon M Kleinberg, Ravi Kumar, Prabhakar Raghavan, Sridhar Rajagopalan, and Andrew S Tomkins. 1999. The web as a graph: Measurements, models, and methods. In *COCOON'99*. 1–17.
- [29] Jérôme Kunegis. 2013. KONECT: the Koblenz network collection. In *WWW '13*. 1343–1350.
- [30] Mohammed Lalou, Mohammed Amin Tahraoui, and Hamamache Kheddouci. 2018. The Critical Node Detection Problem in networks: A survey. *Comput. Sci. Rev.* 28 (2018), 92–117.
- [31] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. <http://snap.stanford.edu/data>.
- [32] Huan Li, Richard Peng, Liren Shan, Yuhao Yi, and Zhongzhi Zhang. 2019. Current Flow Group Closeness Centrality for Complex Networks?. In *WWW 2019*. 961–971.
- [33] Edo Liberty and Maxim Sviridenko. 2017. Greedy Minimization of Weakly Supermodular Set Functions. In *APPROX/RANDOM*, Vol. 81. 19:1–19:11.
- [34] Yang Liu, Xiaoqi Chen, Xi Wang, Zhen Su, Shiqi Fan, and Zhen Wang. 2025. Efficient Continuous Network Dismantling. *IEEE Trans. Syst. Man Cybern. Syst.* 55, 2 (2025), 976–989.
- [35] Siqi Ma, Weixin Zeng, Weidong Xiao, and Xiang Zhao. 2024. Dismantling complex networks with graph contrastive learning and multi-hop aggregation. *Inf. Sci.* 676 (2024), 120780.
- [36] Daniel Marx. 2006. Parameterized graph separation problems. *Theor. Comput. Sci.* 351, 3 (2006), 394–406.
- [37] Robert Meusel, Sebastiano Vigna, Oliver Lehmer, and Christian Bizer. 2014. Graph structure in the web - revisited: a trick of the heavy tail. In *WWW '14, Companion Volume*. 427–432.
- [38] Flaviano Morone and Hernán A Makse. 2015. Influence maximization in complex networks through optimal percolation. *Nature* 524, 7563 (2015), 65–68.
- [39] Flaviano Morone, Byungjoon Min, Lin Bo, Romain Mari, and Hernán A Makse. 2016. Collective influence algorithm to find influencers via optimal percolation in massively large social media. *Sci. Rep.* 6, 1 (2016), 30062.
- [40] Adilson E Motter. 2004. Cascade control and defense in complex networks. *Phys. Rev. E* 93, 9 (2004), 098701.
- [41] Adilson E Motter and Ying-Cheng Lai. 2002. Cascade-based attacks on complex networks. *Phys. Rev. E* 66, 6 (2002), 065102.
- [42] George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. 1978. An analysis of approximations for maximizing submodular set functions - I. *Math. Program.* 14, 1 (1978), 265–294.
- [43] Garry Robins and Philippa Pattison. 2001. Random graph models for temporal processes in social networks. *J. Math. Sociol.* 25, 1 (2001), 5–41.
- [44] Shuichi Sakai, Mitsunori Togasaki, and Koichi Yamazaki. 2003. A note on greedy algorithms for the maximum weighted independent set problem. *Discret. Appl. Math.* 126, 2–3 (2003), 313–322.
- [45] Siqian Shen and J Cole Smith. 2012. Polynomial-time algorithms for solving a class of critical node problems on trees and series-parallel graphs. *Networks* 60, 2 (2012), 103–119.
- [46] Siqian Shen and J. Cole Smith. 2012. Polynomial-time algorithms for solving a class of critical node problems on trees and series-parallel graphs. *Networks* 60, 2 (2012), 103–119.
- [47] Siqian Shen, J. Cole Smith, and Roshan Goli. 2012. Exact interdiction models and algorithms for disconnecting networks via node deletions. *Discret. Optim.* 9, 3 (2012), 172–188.
- [48] Marco Di Summa, Andrea Grosso, and Marco Locatelli. 2012. Branch and cut algorithms for detecting critical nodes in undirected graphs. *Comput. Optim. Appl.* 53, 3 (2012), 649–680.
- [49] Alexander Veremyev, Vladimir Boginski, and Eduardo L. Pasiliao. 2014. Exact identification of critical nodes in sparse networks via new compact formulations. *Optim. Lett.* 8, 4 (2014), 1245–1259.
- [50] Alexander Veremyev, Oleg A. Prokopyev, and Eduardo L. Pasiliao. 2014. An integer programming framework for critical elements detection in graphs. *J. Comb. Optim.* 28, 1 (2014), 233–273.
- [51] Jiazheng Zhang and Bang Wang. [n.d.]. NIRM. <https://github.com/JiazhengZhang/NIRM>.
- [52] Jiazheng Zhang and Bang Wang. 2022. Dismantling Complex Networks by a Neural Model Trained from Tiny Networks. In *CIKM*. 2559–2568.
- [53] Jiazheng Zhang and Bang Wang. 2023. Encoding Node Diffusion Competence and Role Significance for Network Dismantling. In *WWW 2023*. 111–121.
- [54] Fengkuangtian Zhu. [n.d.]. ComplexCi. <https://github.com/zhfkt/ComplexCi>.

A Theoretical Proofs

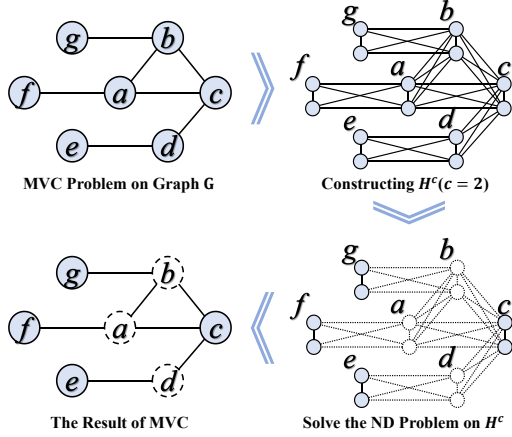
A.1 Theoretical Analysis of the Problem

First, the ND problem is NP-complete.

LEMMA 2. *Given a graph G , a threshold c , and a budget b , the decision problem of ND is to determine whether a vertex cut V' with $|V'| \leq b$ exists that satisfies $|V(MCC(G, V'))| \leq c$. The decision problem of ND is NP-complete [9].*

Moreover, we establish the inapproximability theorem for ND.

THEOREM 8. *Based on the Unique Games Conjecture [26], the ND problem is NP-hard to be approximated within $2 - \epsilon$ for any $\epsilon > 0$.*

Figure 6: An Example of Constructing H^c from G

PROOF. For any graph G and a given threshold c , we construct the graph H^c as follows:

- $V(H^c) = \{(v, i) \mid v \in V(G) \wedge i \in [c]\}$;
- $E(H^c) = \{((u, i), (v, j)) \mid (u, v) \in E(G) \vee u = v\}$.

For any Dismantling $ND(H^c, c)$ of H^c that satisfies the threshold c , we can construct a vertex cover of G as follows. Let $\mathcal{V} \subseteq H^c \setminus ND(H^c, c)$ be formed by randomly sampling one node from each connected component of $H^c \setminus ND(H^c, c)$. Then the set $\{v \in V(G) \mid \forall i, (v, i) \notin \mathcal{V}\}$ is a vertex cover of G . Meanwhile, any vertex cover $VC(G)$ of G naturally corresponds to a valid Dismantling of H^c under threshold c . Specifically, the set $VC(G) \times [c]$ is a feasible Dismantling of H^c subject to c .

In summary, for a given threshold c , there exists a mapping f from a Dismantling V' of H^c to a vertex cover of G such that $|f(V')| \leq |V'|/c$, and for every possible $|f(V')|$, there exists some corresponding V' achieving equality. Thus, assuming the algorithm for ND problem outputs V' that satisfies $|V'| < 2 \cdot |V^*|$, where V^* is the optimal solution to the problem, we have $|f(V')| < 2 \cdot |f(V^*)|$, where $f(V^*)$ is the optimal solution to the MVC problem. Since MVC is NP-hard to be approximated within $2 - \epsilon$ if UGC holds [27], the ND problem is also NP-hard to be approximated within $2 - \epsilon$.

In Fig. 6, we provide an example of constructing H^c from G . $\square$

Clearly, the function f defined in ND is non-increasing and not supermodular; e.g., consider two nodes $u, v \in G$: If $\{v\}$ is not a vertex cut, but $\{u, v\}$ is, then selecting u amplifies the benefit of selecting v . However, if $\{v\}$ is a vertex cut, selecting u diminishes the benefit of selecting v . According to the discussions on approximate submodularity/supermodularity in [11, 33], for ND, Nemhauser Greedy (NG) algorithm [42], i.e. Alg. 7 has the following approximation ratio guarantee, where the NG algorithm repeatedly selects $v = \arg\min_{v \in G \setminus V'} f(V' \cup \{v\})$ to remove from G and add into V' .

THEOREM 9. Let V'^{NG} be the result of Alg. 7, and let OPT be the cardinality of the optimal solution to the ND problem. We have

$$|V'^{NG}|/|OPT| \leq 1 + \frac{\log(n/2)}{\gamma_{V'^{NG}, |OPT|}(n - f)},$$

in which γ is the submodularity ratio defined in [11].

Algorithm 7: Nemhauser Greedy for ND problem

Input : a graph $G = (V, E)$, an integer c
Output : the vertex cut V' with $|MCC(G, V')| \leq c$

```

1 Initialize  $V' \leftarrow \emptyset$ ;
2 while  $|V'| < n$  do
3   if  $f(V') \leq c$  then
4     break;
5   /*  $f(V') = |MCC(G, V')|$  */
6    $v \leftarrow \arg\min_{v \in V(G)} f(V' \cup \{v\})$ ;
7    $V' \leftarrow V' \cup \{v\}, G \leftarrow G \setminus \{v\}$ ;
8 return  $V'$ ;

```

PROOF. [11] provided the following result with respect to the approximation performance of the NG algorithm for the Minimum Cover problem on non-negative, monotone, integer-valued set functions:

LEMMA 3. Let $g(\cdot)$ be a nonnegative, monotone, integer-valued set function. For any given C , Alg. 7 can give an output V' of the problem $\min\{|V'| : V' \subseteq V \wedge g(V') \geq C\}$, that satisfies

$$|V'| \leq \left(1 + \frac{1}{\gamma_{V', |OPT|}(g)} \log\left(\frac{\max_{x \in V} g(x)}{\gamma_{0, |OPT|}(g)}\right)\right) \cdot |OPT|,$$

where OPT is the optimal result and γ is the submodularity ratio.

Since the set function $f(\cdot)$ corresponding to the ND problem is non-increasing monotonically and the constraint of the ND problem is $f(V') \leq c$ (rather than $f(V') \geq c$), we construct a new function $f'(V') = n - f(V')$. Clearly, $f'(\cdot)$ satisfies the conditions required by the lemma, and under this transformation, the constraint of the ND problem becomes $f'(V') \geq n - c$. Therefore, we can employ the lemma to establish Thm. 9. $\square$

Intuitively, Thm. 9 reveals that the approximation ratio of the ND algorithm is bounded by a value related to the submodularity ratio and the gradient of the objective function (which for the ND problem is $n/2$). The submodularity ratio “captures how close the function is to being submodular.”

Despite the approximation guarantee, the NG algorithm is rarely applied directly to ND because of its low efficiency. Specifically, computing $f(V' \cup \{v\})$ requires evaluating the impact of removing any nodes on the current connected components. Computing (or incrementally maintaining) such impacts is extremely costly during iterative removal of nodes, resulting in a time complexity close to $O(|V'| \cdot n \cdot m)$, which is prohibitive for large-scale graphs.

A.2 Theoretical Analysis of the Algorithm

Proof of Thm. 1. Lines 10 - 18 of Alg. 1 complete the maintenance of the adjacency list of G^* . This part enumerates each node in $N(V_1, G^*)$ as well as nodes in $\bigcup_{V_2 \in N(V_1, G^*) \wedge V_1 \notin V'} N(V_2, G^*)$. For edge $(V_1, V_2) \in E(G^*)$, once V_1 is inserted into the sketch graph, once the size of the connected component (where V_1 is located) increases, (V_1, V_2) is enumerated in the above process, until V_2 is also inserted. Thus, the time complexity of this part is $O((c \cdot m)$, where c is the input.

Lines 19 - 21 of Alg. 1 update $score(\cdot)$ of each node affected by the insertion of node v and adjust the bucket list. Due to Lemma 1, the time complexity of this part is $O((c \cdot n)$, where c is the input.

In addition, lines 1 - 6 of Alg. 1 run in $O(m)$ time, and the Disjoint-set operations implied by the algorithm run in $O(\alpha(c) \cdot m)$ time. Combining the above three parts, the total time complexity of Alg. 1 is $O(c \cdot m)$.

Proof of Thm. 3. Each subtree in the tree constructed by the process of Alg. 4 corresponds to a certain component $V_1 \in G^*$ constructed by line 13 of Alg. 1. Due to the definition of the component graph G^* , each such subtree represents a connected subgraph of G . Furthermore, consider an edge $(u, v) \in E(G)$ where node u is removed from V' before node v during the execution of IG+ ($G, b = 0$). Let $u \in V_2$ and $v \in V_1$, where $V_1, V_2 \in V(G^*)$. Then, it always holds that $V_2 \in N(V_1, G^*)$ at the time just before v is removed from V' . This implies that u belongs to the subtree rooted at v and v is an ancestor of u in the constructed tree. Therefore, the resulting tree satisfies all the conditions of a CCRT.

Proof of Property 2. Let $s^*(v)$ denote the value of $score(v)$ at the moment just before node v is inserted into $G[V \setminus V']$, let $b_1 = |IG+(G, c)|$, i.e., the result of IG+. Due to the monotonicity of $s^*(\cdot)$ with respect to the enumeration order of the node, there are b_1 nodes in G that satisfy $s^*(\cdot) \leq c$.

Let $b_2 = |CCRT_G.solve(c)|$. Since $CCRT_G.solve(c)$ removes a node v if and only if $|V(CCRT_G.subtree_v)| > c$, there are b_2 subtrees in $CCRT_G$ whose sizes are no larger than c .

Due to the correspondence between Alg. 2 and Alg. 4, assume $CCRT_G$ is the index generated by Alg. 4. When the IG+ algorithm executes line 18, the current V_1 and v satisfy $s^*(v) = |V(CCRT_G.subtree_v)|$, i.e., there exists a one-to-one correspondence between $s^*(\cdot)$ and the subtrees in $CCRT_G$ except for $CCRT_G.subtree_{vr}$.

Therefore, we conclude that $b_1 = b_2$.

Proof of Thm. 4. First, for all $w \in V(G)$, $CCRT_{G'}.subtree(w)$ in the original graph G either forms a connected subgraph or consists of two connected components, with u and v belonging to different components. In the former case, the subtree remains connected in G' . In the latter case, due to the newly inserted edge (u, v) , the subtree becomes connected in G' . Therefore, $CCRT_{G'}$ satisfies condition (ii). Second, for all $w \in V(G)$, it holds that $CCRT_G.subtree(w) \subseteq CCRT_{G'}.subtree(w)$, and a similar inclusion relationship holds for the nodes' ancestors. Meanwhile, the two endpoints of the newly inserted edge have a parent-child relationship in $CCRT_{G'}$. Therefore, $CCRT_{G'}$ satisfies condition (iii). In addition, vr remains the root of the tree. Therefore, the correctness of the algorithm is proved.

Proof of Thm. 5. The dominating cost is from the traversal of the ancestors of u and v , which ends at $lca(u, v)$. Meanwhile, the time complexity for traversing each node and modifying the tree structure is constant. Therefore, the overall time complexity is $O(dep(u) + dep(v) - 2dep(lca(u, v)))$.

Proof of Thm. 6. First, for all node pairs (u, v) s.t. $u \in CCRT_{G'}.cld_v$, there must exist a node $w \in CCRT_{G'}.subtree_u$ s.t. $(v, w) \in E(G')$. Therefore, all the subtrees of $CCRT_{G'}$ are connected subgraphs of G' , which means condition (ii) of Def. 6 holds. Next, if $(u, v) \in E(G)$ and v is an ancestor of u in $CCRT_G$, as long as (u, v) remains in $E(G')$, lines 12–17 of Alg. 6 ensure that v is still an ancestor of the subtree containing u . Therefore, $CCRT_{G'}$ satisfies condition (iii). In addition, vr remains the root of the tree. Therefore, the correctness of the algorithm is proved.

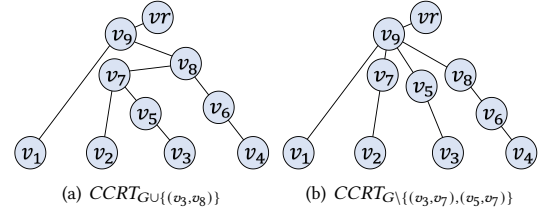


Figure 7: Algorithm Examples

Proof of Thm. 7. The dominating cost is from the traversal of the CCRT, during which each node enumerates its neighbors in the graph and performs union operations of a disjoint set. Therefore, the overall time complexity is $O((n + m)\alpha(n))$, where $\alpha(n)$ is the time complexity of each operation of the disjoint set.

B Algorithm Examples

In this section, we provide examples for CCRT-ins (i.e., Alg. 5) and CCRT-rem (i.e., Alg. 6) following Fig. 7.

Example 1. When inserting (v_3, v_8) , Alg. 5 initializes by setting $u = v_3, v = v_8$ (Line 1). In the first round, the algorithm sets $w = u = v_3, u = CCRT.pa_u = v_5$, and removes w from $CCRT$ (Lines 8 - 11). In the second round, the algorithm links $w = v_3$ with $u = v_5$ (Lines 4 - 7), sets $w = u = v_5, u = CCRT.pa_u = v_7$, and removes w from the $CCRT$ (Lines 8 - 11). Next, the algorithm repeats the above steps, setting $w = u = v_7$ and $u = CCRT.pa_u = v_9$. Finally, the algorithm links $w = v_7$ with $v = v_8$.

Example 2. When removing (v_3, v_7) and (v_5, v_7) , Alg. ?? begins a depth-first search (DFS) from $CCRT.subtree_{vr}$. Once the algorithm finishes traversing $CCRT.subtree_{v_7}$, since $(v_3, v_7), (v_5, v_7) \notin G'$, it no longer connects $CCRT.subtree_{v_5}$ to v_7 (Lines 9 & 12). Then, after completing the traversal of $CCRT.subtree_{v_9}$, since $(v_5, v_9), (v_2, v_9) \in G'$, the algorithm connects $CCRT.subtree_{v_5}$ and $CCRT.subtree_{v_7}$ to v_9 respectively (Lines 12 - 17).

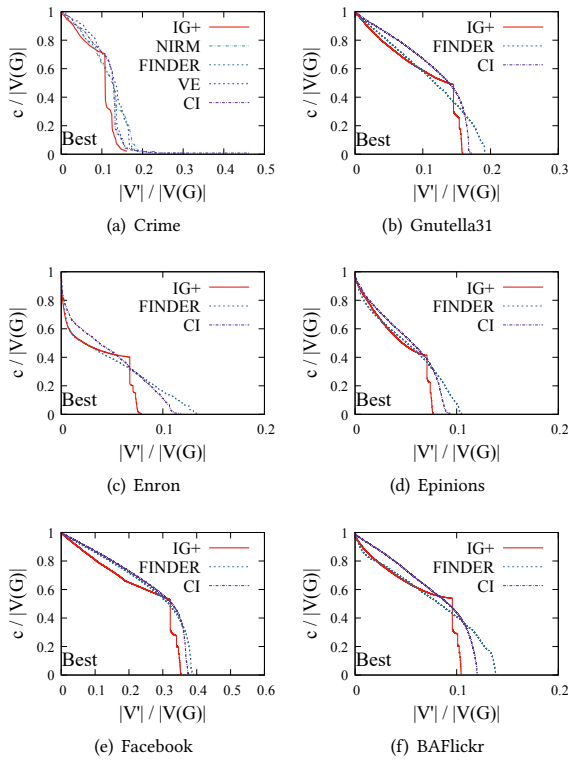
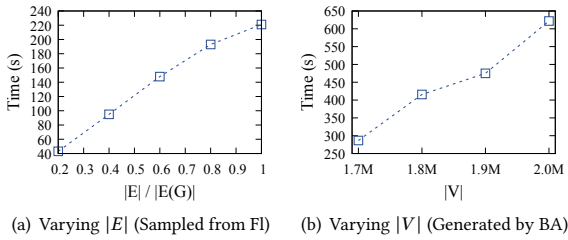
C Experiment Supplementation

As shown in Tab. 3, the solutions provided by IG+ significantly outperform all the baseline methods when we set a relatively small threshold. For example, on the Flickr dataset, IG+ can reduce the size of MCC to 1% of its original size using a budget 1.7x smaller than FINDER (119,808 vs. 201,461). In addition, Figs. 8, 10, and 11 are the supplementations of the experimental results of Exp-1 and Exp3. Note that although we report the result of CI on the BAFlickr dataset in Fig. 8, obtaining this result took about 23 hours, which exceeds our preset time limit (i.e., 12 hours).

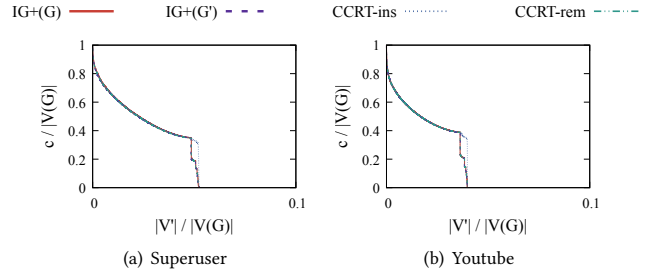
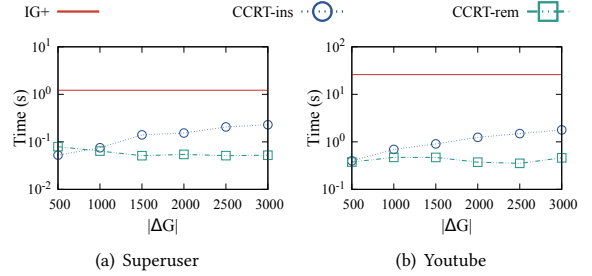
Exp-4 Scalability of IG+. We further evaluate the scalability of IG+ on the largest real-world dataset, Flickr, as well as on synthetic datasets. Specifically, for varying edge sizes, we sampled subgraphs of Flickr with edge counts $|E| \in \{0.2 \cdot |E(G)|, 0.4 \cdot |E(G)|, 0.6 \cdot |E(G)|, 0.8 \cdot |E(G)|, 1.0 \cdot |E(G)|\}$ to assess performance under different edge scales. For varying node sizes, we generated synthetic scale-free networks based on Flickr using the BA model, where the average degree was preserved while progressively expanding the number of nodes to 1.7M, 1.8M, 1.9M, and 2.0M. Notably, the dataset

Table 3: Size of V' for Different Algorithms at Different Threshold c (Bold Is the Optimal Result)

Dataset	$c = 20\% \cdot V(G) $					$c = 10\% \cdot V(G) $					$c = 1\% \cdot V(G) $				
	VE	NIRM	CI	FINDER	IG+	VE	NIRM	CI	FINDER	IG+	VE	NIRM	CI	FINDER	IG+
Cr	138	140	110	115	104	140	144	127	126	112	190	192	180	185	162
HI	438	413	404	408	347	465	457	425	441	358	558	663	469	554	358
Di	-	4167	3770	3998	3544	-	4617	3832	4182	3622	-	4776	3989	4334	3691
Gn	-	-	10321	10962	9625	-	-	10473	11564	9817	-	-	10689	12060	9911
En	-	-	2951	3099	2380	-	-	3400	3869	2484	-	-	3724	4465	2583
Ep	-	-	6280	6707	5614	-	-	6519	7381	5703	-	-	6967	7914	5810
Fa	-	-	23212	23907	21675	-	-	23423	24233	22189	-	-	23790	24667	22370
Yo	-	-	49367	55286	43566	-	-	52796	63936	44577	-	-	55315	71082	45128
Fl	-	-	140515	161143	115911	-	-	150026	182921	118894	-	-	164296	201461	119808
BA	-	-	231344	254510	204995	-	-	237301	272478	207512	-	-	240632	276574	208618

**Figure 8: $|V'|$ of Different Algorithms when Threshold c Changes (Supplementary Results for Exp-1)****Figure 9: The Scalability of IG+ on Flickr and BAFlickr**

with $|V| = 2.0M$ corresponds to the BAFlickr dataset introduced

**Figure 10: Results after Inserting/Removing E^* (Supplementary Results for Exp-3)****Figure 11: Maintenance Time with Different $|\Delta G|$ (Supplementary Results for Exp-3)**

earlier. As shown in Fig. 9, IG+ exhibits near-linear scalability with respect to edge and node count.

Exp-5 Effectiveness of Optimizations. We also compare *IG+ with its variants, *IG and *IG-B, to validate the effectiveness of our optimizations, where *IG/*IG-B represents the exhaustive version of IG, equipped with the heap/bucket list that computes results for all values of c . Fig. 3 reports the time cost of these methods. By comparing the running times of *IG, *IG-B, and *IG+, it is clear that either using the bucket list on IG or applying the delayed update optimization on *IG-B can speed up the computation on all the datasets. *IG+ achieves a speedup of 2x to 10x on all datasets, compared with *IG. The speedup is larger on a larger dataset, e.g., *IG+ is about 6.79x faster than *IG (212 s versus 1440 s) on Flickr. Although *IG (resp. *IG-B and *IG+) essentially computes the result of every possible input c for IG (resp. IG-B and IG+), the effectiveness of optimizations is similar for computing with a fixed c .